



Deploy Your Django App On IBM Cloud Foundry

Estimated time needed: 20 minutes

Learning Objectives

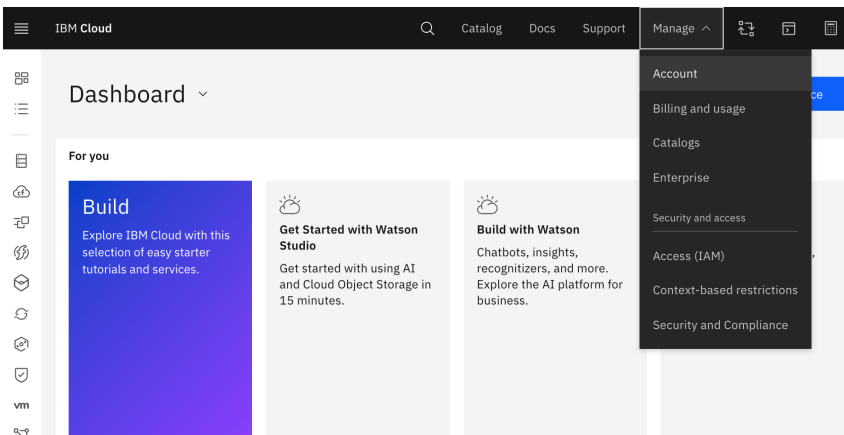
- Understand how to prepare your Django app for Cloud Foundry Deployment
- Deploy your app on IBM Cloud Foundry

Pre-requisites

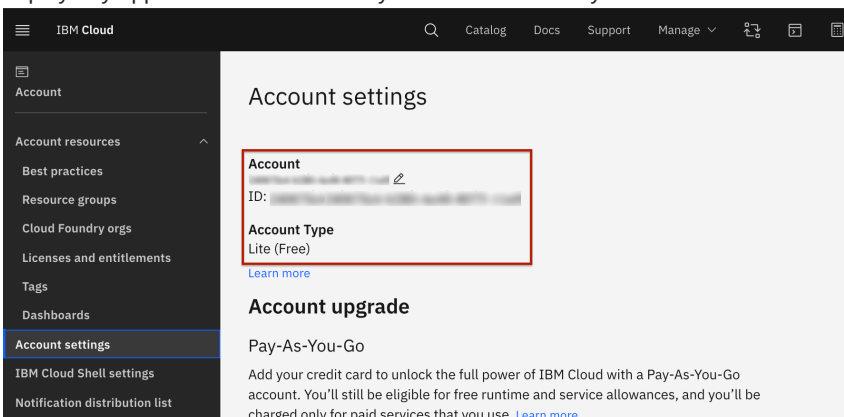
You will need an IBM Cloud account to do this lab. If you have not created one already, click on this [link](#) and follow the instructions to create an IBM Cloud account.

Task 1: Create Space in IBM Cloud

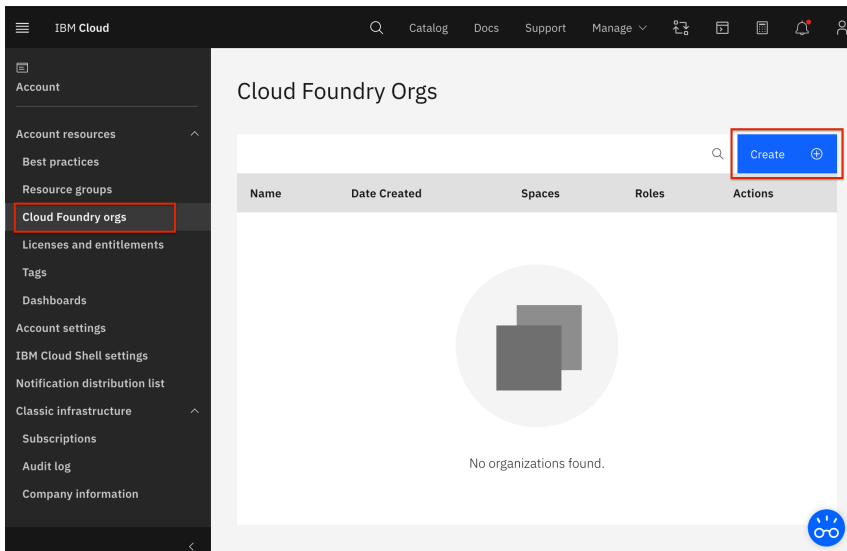
1. Click [here](#) to login to IBM Cloud with your credentials.
2. Click on **Manage** button on the top menu, and go to **Account**.



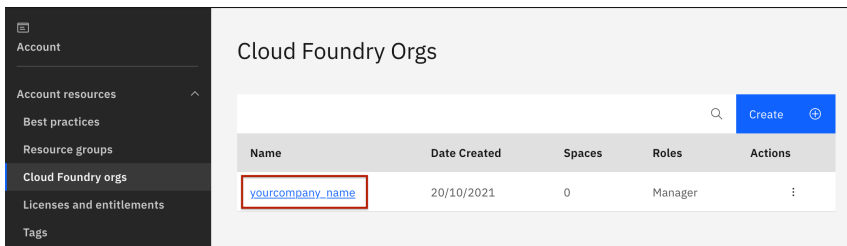
3. Select **Account settings** on the left menu and you should see your account type here. With a lite account, you can deploy any app on IBM Cloud Foundry with 256MB memory for free.



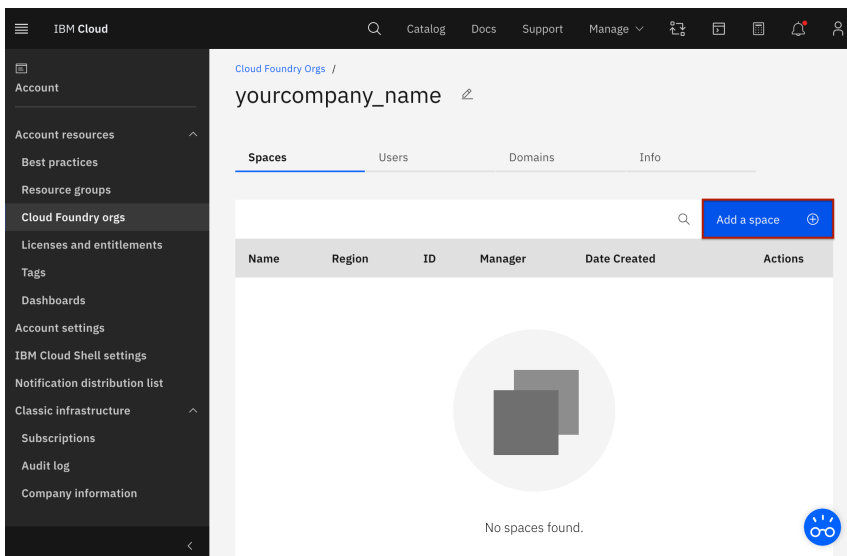
4. Click on **Cloud Foundry org** menu item. You will see the list of Cloud Foundry orgs. If no Cloud Foundry org entry on the list, click **Create** button on the right to create a new one with a desired org name.



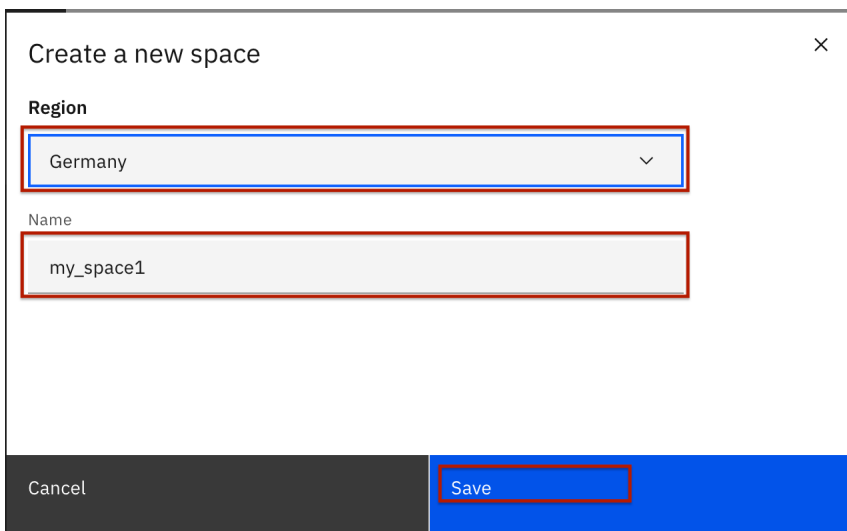
5. Click the created Cloud Foundry org entry.



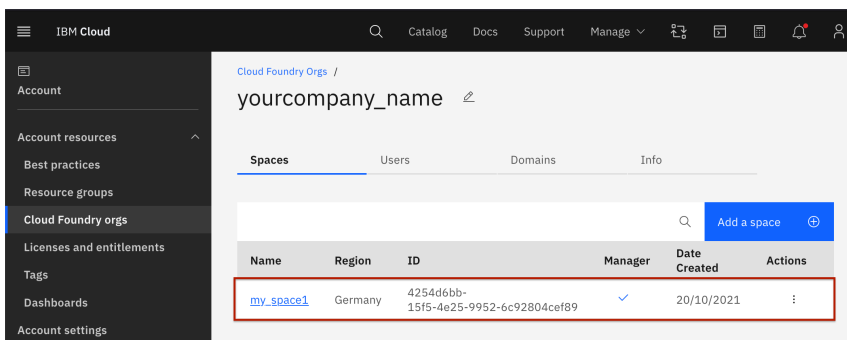
6. Click on **Add Space** to create a space under the org.



7. You need to select a region close to you. In the image below, **Germany** is chosen. Set a name for the space and click on **Save**.



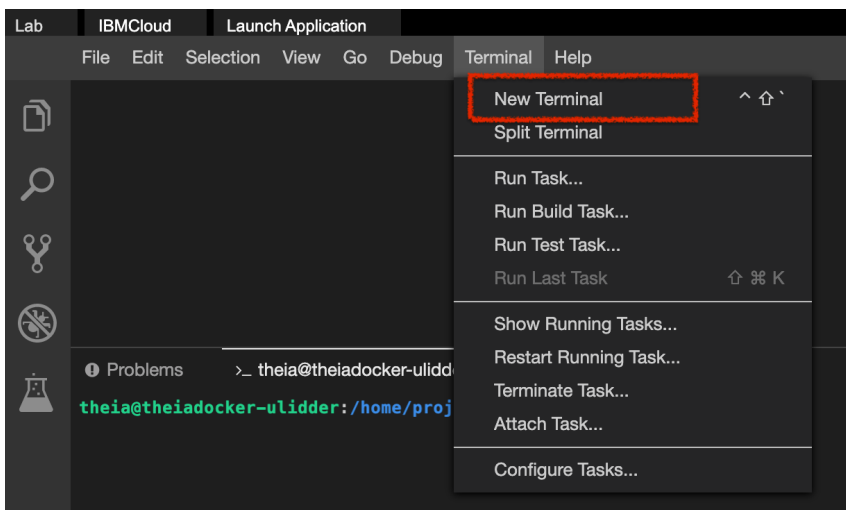
8. Verify that the **Space** is created.



Task 2: Communicate with IBM Cloud via CLI

You can communicate with IBM Cloud via a command-line interface (CLI) which is pre-installed for your Theia image.

1. If the terminal was not open, go to **Terminal > New Terminal**



2. Make sure your current Theia directory is **/home/project**.

```
cd /home/project
```

3. Let's first login with IBM Cloud in Theia terminal console

```
ibmcloud login -u youremail -p yourpassword
```

```
API endpoint: https://cloud.ibm.com
Region: us-south
Authenticating...
OK

Targeted account Yan Luo's Account (150e4fa641a34f719c488f3caf7701ba)

API endpoint: https://cloud.ibm.com
Region: us-south
User: ibmyluo@gmail.com
Account: Yan Luo's Account (150e4fa641a34f719c488f3caf7701ba)
Resource group: No resource group targeted, use 'ibmcloud target -g RESOURCE_GROUP'
CF API endpoint:
Org:
Space:
theia@theiadocker-ylo: /home/project$
```

4. For IBM Cloud Foundry communication, you need to install a CLI tool.

```
ibmcloud cf install
```

5. Then select the IBM Cloud org and space you created in the previous task.

```
ibmcloud target -o yourorg -s yourspace
```



The parameter after `-o` is your IBM Cloud Foundry org name and the parameter after `-s` is a space name under that org.

6. You should see your **Org** and **Space** in the terminal:

```
API endpoint:      https://cloud.ibm.com
Region:           us-south
User:             ibmyluo@gmail.com
Account:          Yan Luo's Account (150e4fa641a34f719c488f3caf7701ba)
Resource group:   No resource group targeted, use 'ibmcloud target -g RESOURCE_GROUP'
CF API endpoint:  https://api.us-south.cf.cloud.ibm.com (API version: 2.156.0)
Org:              ibmyluo@gmail.com
Space:            dev
theia@theiadocker-yluo:/home/project$
```

7. Then you could use `ibmcloud cf` CLI to check your current available domain names, which will be used for building up the publicly accessible URL of your app.

```
ibmcloud cf domains
```



You should see available domains in your org:

```
Getting domains in org ibmyluo@gmail.com as ibmyluo@gmail.com...
name                    status  type  details
mybluemix.net           shared
apic.mybluemix.net      shared
us-south.cf.appdomain.cloud shared
apps.internal           shared  internal
```

8. If you see an error like the following, please make sure `ibmcloud cf install` was ran successfully.

```
Invoking 'cf domains'...
```

```
No API endpoint set. Use 'cf login' or 'cf api' to target an endpoint.
FAILED
```



Now, you have setup an IBM Cloud account. Let's prepare our Django app for deployment.

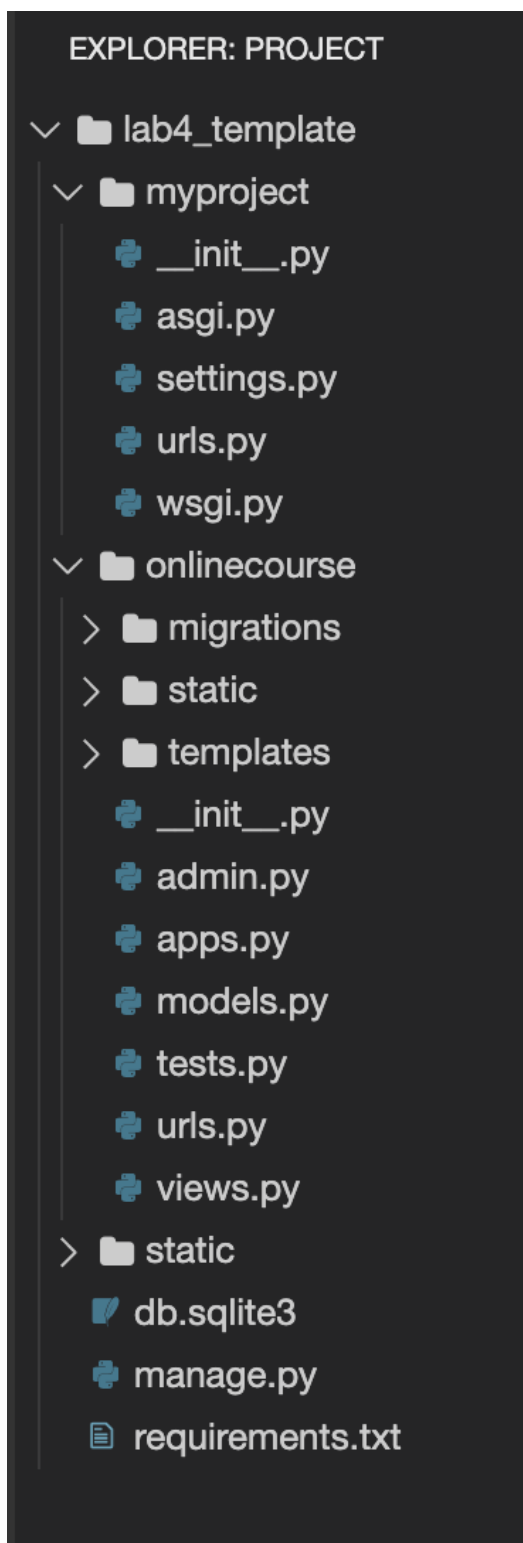
Task 3 - Import an **onlinecourse** App Template and Database

1. Run the following command-lines to download a code template for this lab

```
wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-CD0251EN-SkillsNetwork/labs/m5_django_advanced/lab4_template.zip"
unzip lab4_template.zip
rm lab4_template.zip
```



Your Django project should look like the following:



2. Change to the project folder.

```
cd lab4_template
```

3. Install the necessary Python packages.

```
python3 -m pip install -U -r requirements.txt
```

4. Perform migrations to create necessary tables:

```
python3 manage.py makemigrations
```

5. Run migration to activate models for **onlinecourse** app.

```
python3 manage.py migrate
```

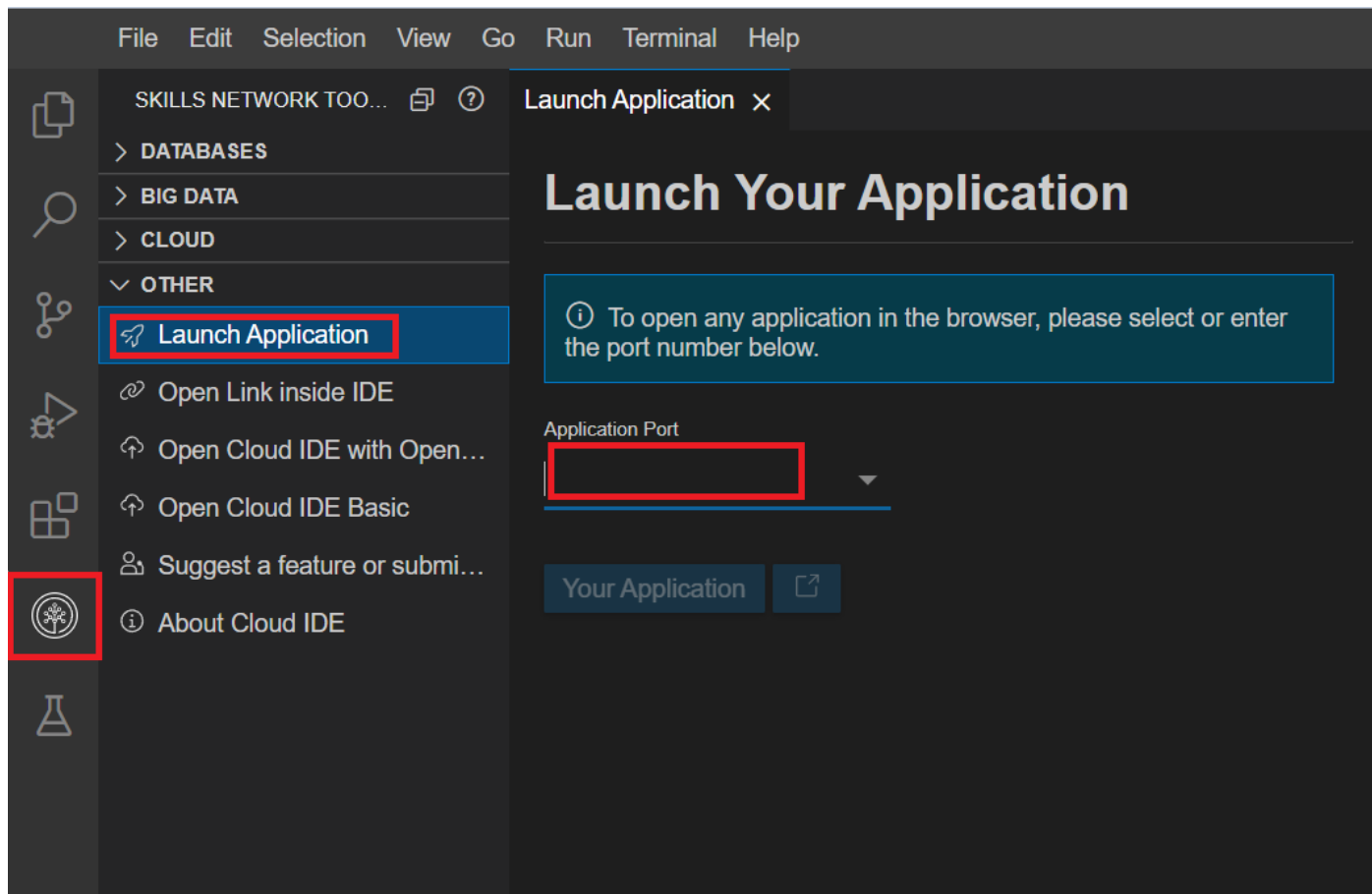


6. Now let's test the imported `onlinecourse` app. Start the local development server.

```
python3 manage.py runserver
```



7. Click on the Skills Network button on the left, it will open the "**Skills Network Toolbox**". Then click the **Other** then **Launch Application**. From there you should be able to enter the port `8000` and launch.



8. When the browser tab opens, add the `/onlinecourse` path and your full URL should look like the following

`https://userid-8000.theiadocker-1.proxy.cognitiveclass.ai/onlinecourse`

Now you should see the `onlinecourse` app started.

Task 4: Deploying a Django App on IBM Cloud Foundry

We need to create some extra deployment-related configuration files to tell IBM Cloud Foundry how to prepare the runtime environments and infrastructure for our apps.

The most important configuration metadata we need to send to IBM Cloud Foundry is the what kinds of runtime environments. For Cloud Foundry, the app runtime is managed by different buildpacks.

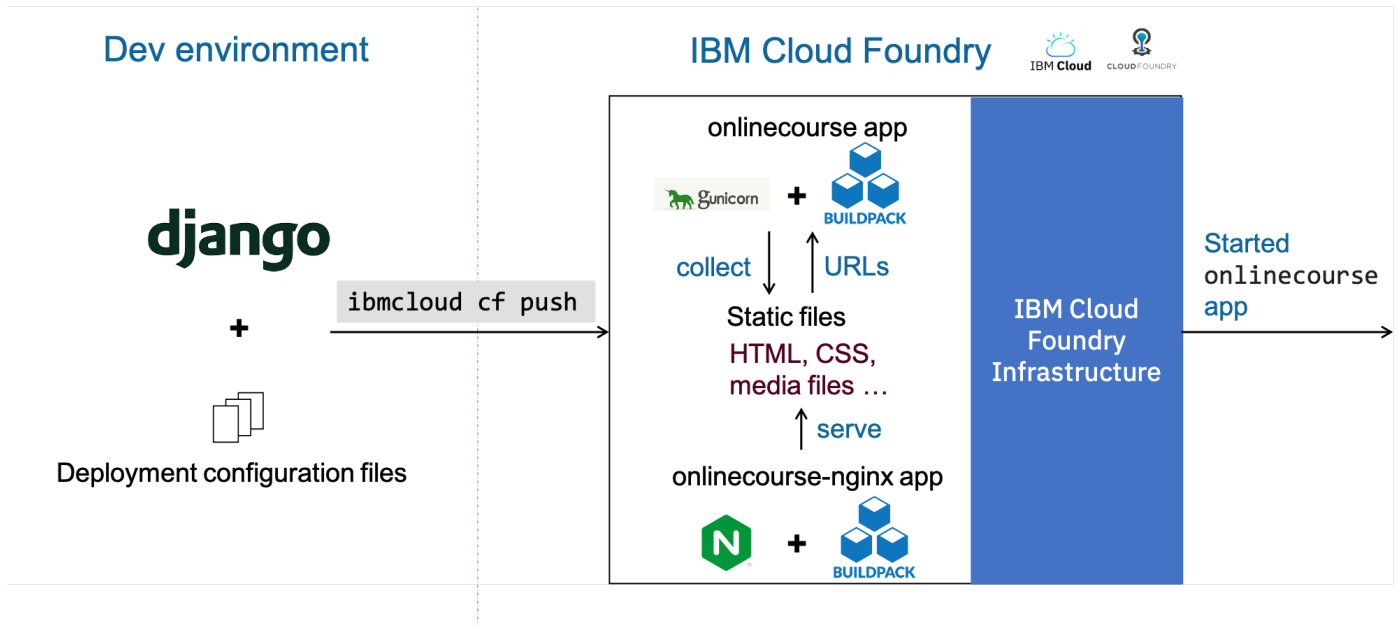
For our Django `onlinecourse` app, we will ask IBM Cloud Foundry to first start a Python runtime instance with web server `gunicorn` started to server our Django app. This runtime instance is managed by a Python buildpack provided by Cloud Foundry. Python buildpack provides a framework and runtime support for our `onlinecourse` app.

In addition to prepare the `gunicorn` server and `onlinecourse` app, Python buildpack also collects the static files in our app such as html template files, CSS, and media files, to the `STATIC_ROOT` directory we specified in `settings.py`.

To serve those static files, we will need another webserver for better load balancing management. Cloud Foundry provides a Staticfile buildpack supported by `nginx` server, which is also a popular web server and widely used for serving static files. Here, we create a staticfile buildpack-based instance to serve static files and provide URLs to our `onlinecourse` Django server.

With both `onlinecourse` and `onlinecourse-static` Cloud Foundry apps running on Cloud Foundry, we will have a fully functional `onlinecourse` Django app that can be accessed via a public URL.

The following flowchart shows the deployment process from high level:



Task 5: Prepare **onlinecourse** App For Deployment

Next, let's prepare our Django app for deployment.

First, in order to run our online course app, we need to create a Procfile file specifying the commands for IBM Cloud Foundry.

1. Create an empty file called **Procfile** under **lab4_template** and adds a command line for **gunicorn** server.

```
web: gunicorn myproject.wsgi
```

It tells IBM Cloud Foundry to start a **gunicorn** webserver and find an application callable declared in a **myproject/wsgi.py** file.

2. Next, we need a manifest file to customize IBM Cloud Foundry push command for starting a deployment pipeline for our app. Create a **manifest.yml** file under **lab4_template** and adds the following configuration entries:

```
applications:
- name: onlinecourse
  routes:
    - route: yourhost.us-south.cf.appdomain.cloud
  memory: 128M
  buildpack: python_buildpack
- name: onlinecourse-nginx
  routes:
    - route: yourhost.us-south.cf.appdomain.cloud/static
  memory: 128M
  buildpack: staticfile_buildpack
```

Manifest file details

- The main part of the manifest file is an application list containing the applications or instances we want IBM Cloud Foundry to launch.
- The first application entry is the online course Django app with a name **onlinecourse**.

We also need to specify its route information which is a public URL that contains the host and domain of the app.

- You can find an available domain name via **ibmcloud cf domains** and choose one for your app.
- Replace **yourhost** with a unique hostname.

```

applications:
- name: onlinecourse
  routes:
    - route: yluo.us-east.mybluemix.net
  memory: 128M
  buildpack: python_buildpack
- name: onlinecourse-nginx
  routes:
    - route: yluo.us-south.cf.appdomain.cloud/static
  memory: 128M
  buildpack: staticfile_buildpack

```

- Then we specify the resource such as the memory heap size of our app. You can have max 256MB for Python runtime with your IBM Cloud lite account but 128MB should be enough for the small `onlinecourse` app.

- We then ask IBM Cloud Foundry to start a Python buildpack to launch a specified Python runtime defined in the `requirements.txt`.

- The second application entry on the list is the `onlinecourse-nginx` app for serving the static files. It specifies route information as well as a staticfile buildpack to start an `nginx` server for serving the static files.

- Finally, with the URL contains host and domain name configured defined, we need to add it into the allowed host in `myproject/settings.py`

- Find `ALLOWED_HOSTS` list in `myproject/settings.py` and append the host using the `route` entry in `manifest.yml`

```
ALLOWED_HOSTS = ['yourhost.us-south.cf.appdomain.cloud']
```

At this point, you just made `onlinecourse` app ready for IBM Cloud Foundry deployment.

Task 6: Deploy the `onlinecourse` App on IBM Cloud Foundry

Lets start deploying `onlinecourse` app.

1. Simply run `ibmcloud cf push` to push the app and start the deployment pipeline. You may ignore any warnings during the deployment process.

2. Once the deployment finishes, you should see following messages in the terminal:

`onlinecourse` Django app started on `gunicorn` server:

Waiting for app to start...

name: onlinecourse
requested state: started
routes: yluo.us-south.cf.appdomain.cloud
last uploaded: Mon 14 Dec 22:03:36 UTC 2020
stack: cflinuxfs3
buildpacks: python

type: web
instances: 1/1
memory usage: 128M
start command: gunicorn myproject.wsgi

	state	since	cpu	memory	disk	details
#0	running	2020-12-14T22:04:04Z	0.0%	51M of 128M	249.3M of 1G	



onlinecourse-nginx Nginx server started:

Waiting for app to start...

name: onlinecourse-nginx
requested state: started
routes: yluo.us-south.cf.appdomain.cloud/static
last uploaded: Mon 14 Dec 22:04:25 UTC 2020
stack: cflinuxfs3
buildpacks: staticfile

type: web
instances: 1/1
memory usage: 128M
start command: \$HOME/boot.sh

	state	since	cpu	memory	disk	details
#0	running	2020-12-14T22:04:47Z	0.3%	21.9M of 128M	8.1M of 1G	



3. Now, you can go to IBM Cloud [Resource list](#) to check the deployed apps.

4. In the Cloud Foundry apps section, you should see that the **onlinecourse** Django app and **onlinecourse-nginx** app are started.

</

5. Click on **onlinecourse** app to check its app details.

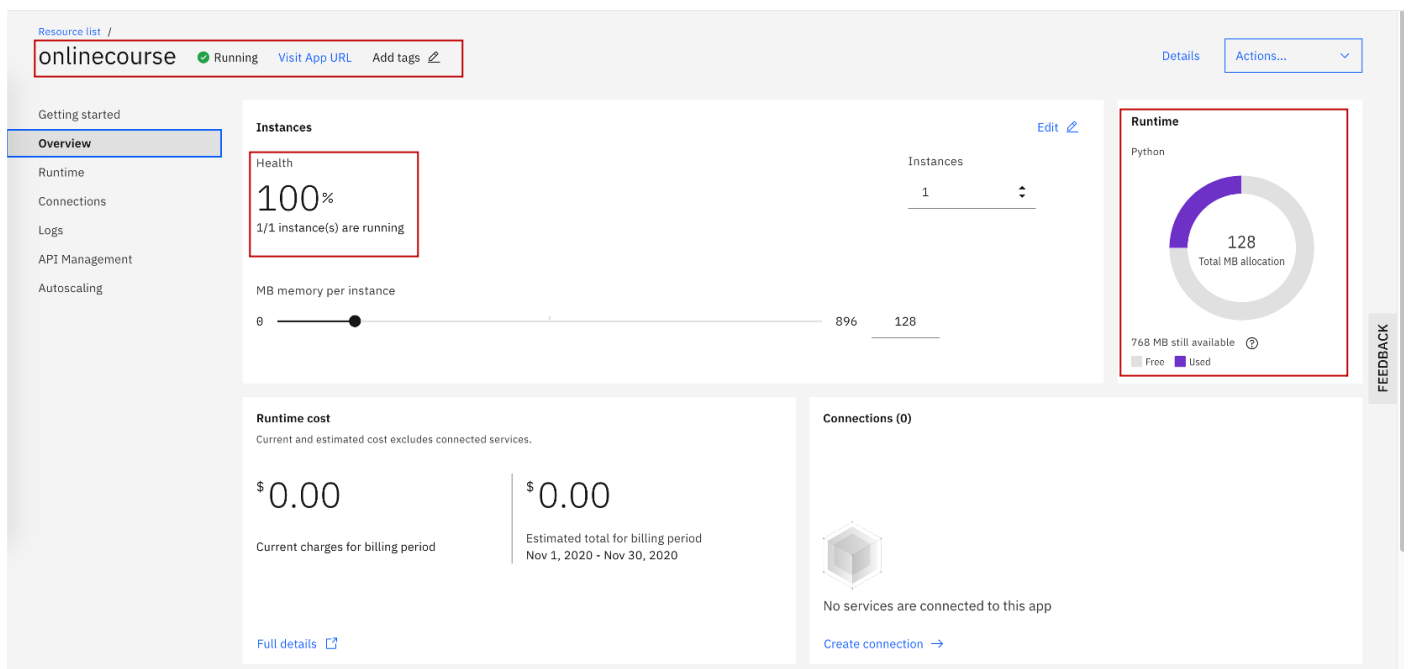
we can check the app's status and URL, as well as its health status and memory usage status.

6. At last, click the **Visit App URL** link and add a **/onlinecourse** path. Your full URL

would look like the following:

yluo.us-south.cf.appdomain.cloud/onlinecourse





7. You may also try to access a static file such as

http://yluo.us-south.cf.appdomain.cloud/onlinecourse/static/media/course_images/django.png which is served by `onlinecourse-nginx` app.

Summary

In this lab, you have learned how to prepare and deploy the `onlinecourse` app on IBM Cloud Foundry. In addition, you have learned that to start a `nginx` server to serve the static files of your `onlinecourse` app on IBM Cloud Foundry.

Author(s)

[Yan Luo]([linkedin.com/in/yan-luo-96288783](https://www.linkedin.com/in/yan-luo-96288783))

Changelog

Date	Version	Changed by	Change Description
14-Dec-2020	1.0	Yan Luo	Initial version created
20-Oct-2021	1.1	Lavanya	Updated as per recent Cloud changes

© IBM Corporation 2021. All rights reserved.